

TEXTBOOK EDITION • PYTHON & DATA ENGINEERING

Python Data Ecosystem & Analytics

NumPy Vectorization, pandas DataFrames, Matplotlib & Banking Analytics Pipeline

TARGET ROLE	Developer (3+ Years Experience)
DEPARTMENT	Strategic Projects & New Age Tech, Bengaluru
PREPARED FOR	Bipin Yadav (SDE II, 4+ YoE)
ESTIMATED STUDY TIME	12 Hours
DIFFICULTY LEVEL	Advanced Core
TEACHING FORMAT	Hinglish Mental Models + Production FinTech

Table of Contents

Chapter 01 — Algorithmic Complexity Analysis (Big-O, Big-Ω, Big-Θ & Amortized Analysis)	Study Time: 35m
Chapter 02 — Python Fundamentals, OOP, Internals & Advanced Concepts	Study Time: 35m
Chapter 03 — NumPy: Numerical Computing & Vectorized Operations	Study Time: 35m
Chapter 04 — pandas: Tabular Data Analysis & Financial DataFrames	Study Time: 35m
Chapter 05 — Matplotlib: Data Visualization & Financial Plotting	Study Time: 35m
Chapter 06 — SQL pandas Rosetta Stone: Translating SQL Knowledge to pandas	Study Time: 35m
Chapter 07 — The Complete Python Data Analysis Workflow: NumPy to pandas to Matplotlib	Study Time: 35m
Chapter 08 — Last-Minute Python Data Ecosystem Cheatsheet	Study Time: 35m

CHAPTER 01

Algorithmic Complexity Analysis (Big-O, Big-Ω, Big-Θ & Amortized Analysis)

PRIORITY

● High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Algorithmic Complexity Analysis (Big-O, Big-Ω, Big-Θ & Amortized Analysis)

1. Why This Matters

In banking and financial backend systems, an algorithm that is $O(N^2)$ instead of $O(N \log N)$ can freeze a settlement engine during peak trading or UPI clearing windows. At IDFC FIRST Bank's Strategic Projects, engineers write code handling tens of thousands of concurrent payment requests. In interviews, stating an algorithm's complexity is not enough: interviewers will ask you to mathematically derive worst-case, average-case, and amortized bounds, identify auxiliary memory allocations, and analyze garbage collection overhead.

2. Prerequisites

- Basic algebra (logarithms, exponents, summation formulas $\sum_{i=1}^n i = \frac{n(n+1)}{2}$).
- Understanding of stack frames and heap allocations in programming languages (Node.js/V8, Python).

3. Concept

Asymptotic Notations

- Big-O (O):** Upper bound. Formal definition: $f(N) = O(g(N))$ if there exist positive constants c and N_0 such that $0 \leq f(N) \leq c \cdot g(N)$ for all $N \geq N_0$. Represents the worst-case scenario.
- Big-Omega (Ω):** Lower bound. $f(N) \geq c \cdot g(N)$ for all $N \geq N_0$. Best-case performance.
- Big-Theta (Θ):** Tight bound. When an algorithm's upper bound and lower bound match: $c_1 \cdot g(N) \leq f(N) \leq c_2 \cdot g(N)$.
- Little-o (o):** Strict upper bound where $f(N)$ grows strictly slower than $g(N)$.

```

flowchart LR
    subgraph Growth["Growth of Common Time Complexities"]
        direction TB
        c1["O(1) - Constant: Hash table lookup, Array indexing"]
        c2["O(log N) - Logarithmic: Binary search, Balanced BST lookup"]
        c3["O(N) - Linear: Single scan, Two Pointers, Sliding Window"]
        c4["O(N log N) - Linearithmic: MergeSort, QuickSort (avg), Timsort"]
        c5["O(N^2) - Quadratic: Nested loops, BubbleSort, Brute force pairs"]
        c6["O(2^N) - Exponential: Naive Fibonacci, Subsets recursion"]
        c7["O(N!) - Factorial: Permutations generation, Traveling Salesperson"]
    end
    c1 → c2 → c3 → c4 → c5 → c6 → c7

```

Amortized Analysis

When an operation is occasionally very expensive but predominantly cheap, the *amortized cost* averages the total cost over a long sequence of operations. **The Accounting Method:** You overcharge cheap operations (e.g., normal array append = 1 unit of work charged as 3 units) and store the surplus as credit. When the expensive resizing operation occurs (copying N elements = N units of work), the accumulated credit pays for the reallocation. Thus, the amortized cost per append is strictly $O(1)$.

4. Simple Example: Linear vs Logarithmic Search

```

# Linear Search: O(N) Time, O(1) Space
def linear_search(arr: list[int], target: int) → int:
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

# Binary Search: O(log N) Time, O(1) Space
def binary_search(sorted_arr: list[int], target: int) → int:
    left, right = 0, len(sorted_arr) - 1
    while left ≤ right:
        mid = left + (right - left) // 2 # Prevents integer overflow
        if sorted_arr[mid] == target:
            return mid
        elif sorted_arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

```

5. Real-World Banking Example: High-Throughput Transaction Deduplication

Suppose an incoming UPI payment batch has $N = 100,000$ transactions. You must verify that no two transactions share the same `idempotency_key`.

- **Brute Force ($O(N^2)$):** Comparing each transaction against every other transaction requires:
 $\frac{100,000 \times 99,999}{2} \approx 5 \times 10^9$ operations $\implies \approx 5 \times 10$ seconds (CPU freeze).

- **Hash Set Optimization ($O(N)$ Time, $O(N)$ Space):** Checking existence in a hash set takes $O(1)$ on average: $\$100,000 \text{ lookups} \approx 0.015 \text{ seconds}$.
- **In-place Sort + Adjacent Scan ($O(N \log N)$ Time, $O(1)$ Auxiliary Space):** Useful when memory is constrained on embedded gateway appliances.

6. Code: Python Dynamic Array Resizing Proof

```
import sys
import time

def demonstrate_amortized_growth():
    """Demonstrates list capacity jumps in Python (CPython dynamic array)."""
    data = []
    prev_size = sys.getsizeof(data)
    print(f"{'Length':<10} | {'Byte Size':<12} | {'New Allocation Triggered?'}")
    print("-" * 50)

    for i in range(25):
        data.append(i)
        current_size = sys.getsizeof(data)
        resized = current_size != prev_size
        if resized:
            print(f"{'len(data)':<10} | {'current_size':<12} | YES (Grown to accommodate elements)")
            prev_size = current_size

if __name__ == "__main__":
    demonstrate_amortized_growth()
```

7. How It Works Internally

CPython & V8 Dynamic Array Growth Mechanics

1. In CPython, `list` is implemented as an array of pointers (`PyObject**`). When the allocated capacity is exhausted, CPython over-allocates using the formula: $\text{new_allocated} = \text{size} + (\text{size} \gg 3) + (\text{size} < 9 ? 3 : 6)$. This provides an approximate geometric growth factor of $\approx 1.125 \times$.
2. In V8 (Node.js/JavaScript), `JSArray` has two modes: **Fast Elements** (contiguous C++ pointer backing stores) and **Dictionary Elements** (hash tables for sparse arrays). For fast elements, resizing multiplies capacity by $1.5 \times$ to $2 \times$.
3. Copying N elements takes $O(N)$, but because reallocation happens geometrically, resizing from 1 to N requires: $1 + 2 + 4 + 8 + \dots + N = 2N - 1 = O(N)$ total copies across N insertions. $\frac{\text{Total Time}}{\text{Operations}} = \frac{O(N)}{N} = O(1)$ amortized per append.

8. Common Mistakes

- Confusing Auxiliary Space with Total Space:** Auxiliary space is temporary working memory *excluding* the input. In MergeSort, total space is $O(N)$, and auxiliary space is $O(N)$. In QuickSort, auxiliary stack space is $O(\log N)$ average, $O(N)$ worst.
- Ignoring String Immutability Overhead:** In Python and JavaScript, strings are immutable. Doing `s += char` inside an N -iteration loop creates a new string each time, turning an intended $O(N)$ operation into $O(N^2)$! Always use `"".join(list)` in Python or an array push + `.join("")` in JS.
- Assuming Hash Table Lookups are Always $O(1)$:** If hash collisions occur (or an adversary triggers hash collision attacks), open-addressed or chained tables degrade to $O(N)$ worst case.

9. Performance / Complexity Matrix

Operation / Algorithm	Best Time	Average Time	Worst Time	Auxiliary Space
Array Access by Index	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Dynamic Array Append	$O(1)$	$O(1)$	$O(N)$	$O(1)$
Hash Table Insert/Lookup	$O(1)$	$O(1)$	$O(N)$	$O(N)$
Binary Search	$O(1)$	$O(\log N)$	$O(\log N)$	$O(1)$
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$
Quick Sort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(\log N)$ avg
Breadth-First Search (BFS)	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$
Depth-First Search (DFS)	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(V)$ stack

10. Interview Questions (Easy to Medium to Hard)

Easy

- Q:** What is the difference between $O(1)$ and $O(\log N)$? Give an example of each in software engineering.

Medium

- Q:** Explain why QuickSort is commonly preferred over MergeSort for arrays in cache-sensitive systems despite QuickSort having a worse worst-case time complexity ($O(N^2)$ vs $O(N \log N)$).

Hard

- **Q:** An algorithm processes a stream of financial transactions. Each incoming item requires checking a min-heap of size K and occasionally rebuilding an auxiliary index of size M . Derive the tight asymptotic bound for N total transactions where $M = \sqrt{N}$ and $K = \log N$.

11. Follow-up Questions from Interviewer

- "What is cache locality, and how does it affect the empirical execution speed of an $O(N)$ array traversal versus an $O(N)$ linked list traversal?"
- "If `Array.prototype.sort()` in Node.js uses Timsort, what is its worst-case space complexity, and why doesn't it use standard in-place QuickSort?"

12. Model Answer (QuickSort vs MergeSort & Cache Locality)

BANKING & FINTECH ARCHITECTURE SCENARIO

Interviewer: "Why does production software often choose QuickSort or Dual-Pivot QuickSort over MergeSort for primitive arrays?"

Model Answer: "While MergeSort guarantees $O(N \log N)$ worst-case time, it requires $O(N)$ auxiliary memory because merging requires a secondary array. In contrast, QuickSort operates strictly in-place with $O(\log N)$ auxiliary call-stack space.

More importantly, QuickSort exhibits superior **spatial locality of reference**. Its partitioning algorithm traverses contiguous memory sequentially, making optimal use of CPU L1/L2 cache lines and minimizing hardware cache misses. MergeSort's auxiliary buffer copying incurs additional memory bus traffic.

However, for reference types or objects (such as JavaScript arrays or Python objects) where elements are pointers and stability is required (preserving relative order of equal keys, like sorting bank transactions by timestamp while preserving category order), Timsort (a hybrid of MergeSort and InsertionSort) is preferred because it guarantees stability and $O(N)$ best-case time on partially sorted data."

13. Practical Exercise

Analyze the time and auxiliary space complexity of each function:

```
# Exercise 1:
def mystery_one(n: int):
    count = 0
    i = n
    while i > 0:
        for j in range(i):
            count += 1
        i //= 2
    return count

# Exercise 2:
def mystery_two(arr: list[int]):
    n = len(arr)
    result = []
    for i in range(n):
        subset = []
        for j in range(i, n):
            subset.append(arr[j])
            result.append(list(subset))
    return result
```

(Solution hint: Mystery 1 uses a geometric series $\sum_{k=0}^{\log_2 n} \frac{n}{2^k} = n(1 + 1/2 + 1/4 + \dots) = O(N)$ time, $O(1)$ space. Mystery 2 generates all subarrays: $O(N^2)$ iterations, but copying subsets makes total time $O(N^3)$ and space $O(N^3)$).

14. Quick Revision

- Big-O denotes worst-case upper bound; Big- Θ denotes tight asymptotic bound.
- Geometric series $N + N/2 + N/4 + \dots = 2N = O(N)$.
- In-place algorithms utilize $O(1)$ auxiliary space beyond input buffers.
- String concatenation inside loops in Python/JS creates $O(N^2)$ memory churning.
- Dynamic array append is $O(1)$ amortized, $O(N)$ worst-case.

15. Interview Checklist

- ☐ Can define Big-O mathematically ($f(N) \leq c \cdot g(N)$).
- ☐ Can explain the accounting/potential method for amortized analysis.
- ☐ Can distinguish auxiliary space from input space.
- ☐ Can explain CPU cache lines and spatial/temporal locality.
- ☐ Can identify hidden $O(N)$ costs in high-level language methods (e.g., `list.insert(0, val)` or `Array.shift()` which shifts all elements).



CHAPTER 01 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Basic algebra (logarithms, exponents, summation formulas $\sum_{i=1}^n i = \frac{n(n+1)}{2}$).
- Understanding of stack frames and heap allocations in programming languages (Node.js/V8, Python).
- Big-O ($O()$): Upper bound. Formal definition: $f(N) = O(g(N))$ if there exist positive constants c and N_0 such that



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 02

Python Fundamentals, OOP, Internals & Advanced Concepts

PRIORITY

● High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Python Fundamentals, OOP, Internals & Advanced Concepts

1. Why This Matters (Interview Perspective)

IDFC FIRST Bank ke job description mein "**Java / Python / C++**" explicitly mentioned hai. Aapka primary backend experience Node.js/Express mein hai aur aapne Python mainly DSA ke liye use kiya hai. Lekin interview mein agar interviewer Python par switch karta hai, toh woh sirf basic loops nahi puchega. Woh puchega: – Python ka memory model (pointers, heap allocations, mutability). – Generators (`yield`) vs Lists memory footprint. – Decorators kaise kaam karte hain under the hood. – Magic methods (`__eq__` , `__repr__` , `__lt__`) custom sorting aur OOP design ke liye. Is module ka goal aapko data scientist banana nahi hai, balki **ek confident 3+ YoE developer ki tarah Python discuss aur code karne ke kabil banana hai**.

2. Python Fundamentals (Hinglish + Code)

Simple Language Mein:

📌 KEY ENGINEERING INSIGHT

"Python mein sab kuch ek **Object** hota hai. Variables koi memory box nahi hote jisme value rakhi hoti hai; variables bas ek **Name Tag (Pointer)** hote hain jo heap par bane object ko point karte hain."

1. Variables, Data Types & Slicing

```
# Numbers & Strings
age: int = 28
balance: float = 45200.50
bank_name: str = "IDFC FIRST Bank"

# Slicing: [start : stop : step]
account_num = "IDFB1000928371"
bank_code = account_num[0:4] # 'IDFB'
last_four = account_num[-4:] # '8371'
reversed_str = account_num[::-1] # Pure string reverse in O(N)
```

2. Lists, Tuples, Sets, Dictionaries

Data Structure	Syntax	Mutability	Duplicates?	Ordered?	Under the Hood
<code>list</code>	<code>[1, 2, 3]</code>	Mutable	Yes	Yes	Dynamic Array (<code>PyObject**</code>)
<code>tuple</code>	<code>(1, 2, 3)</code>	Immutable	Yes	Yes	Fixed-size array (Cache friendly)
<code>set</code>	<code>{1, 2, 3}</code>	Mutable	No	No	Hash Table (Keys only, $O(1)$ avg)
<code>dict</code>	<code>{"a": 1}</code>	Mutable	No (Keys)	Yes (3.7+)	Compact Hash Table ($O(1)$ avg)

```
# Unpacking and *args, **kwargs
def process_transaction(txn_id: str, *amounts, **metadata):
    """
    *amounts captures positional args as a tuple
    **metadata captures keyword args as a dictionary
    """
    total = sum(amounts)
    channel = metadata.get("channel", "UPI")
    print(f"Txn: {txn_id}, Total: ₹{total}, Channel: {channel}")

process_transaction("TXN_101", 500, 1200, 350, channel="IMPS", merchant="Swiggy")
# Output: Txn: TXN_101, Total: ₹2050, Channel: IMPS
```

3. Comprehensions (List, Dict, Set)

KEY ENGINEERING INSIGHT

"Loop likhne ke bajaye ek line mein readable aur fast collection banana."

```
# List Comprehension (Filtered & Transformed)
raw_amounts = [1500, -200, 8000, 0, 12000]
credits_only = [amt for amt in raw_amounts if amt > 0]
# [1500, 8000, 12000]

# Dict Comprehension: Customer ID to normalized account map
accounts = [("CUST_101", "SAVINGS"), ("CUST_102", "CURRENT")]
acc_dict = {cust_id: acc_type for cust_id, acc_type in accounts}
# {'CUST_101': 'SAVINGS', 'CUST_102': 'CURRENT'}

# Set Comprehension: Unique merchant categories
merchants = ["Swiggy", "Zomato", "Swiggy", "Amazon", "Uber"]
unique_merchants = {m.upper() for m in merchants}
# {'SWIGGY', 'ZOMATO', 'AMAZON', 'UBER'}
```

3. Python OOP (Object-Oriented Programming)

Core Pillars:

1. **Encapsulation:** Data aur methods ko ek unit (class) mein bind karna. Private members convention se `_single_underscore` (internal) ya `__double_underscore` (name mangling) hote hain.

2. **Inheritance:** Parent class ki functionality reuse karna.
3. **Polymorphism:** Same method name different classes mein alag behave kare (Duck Typing: *"If it walks like a duck and quacks like a duck, it's a duck"*).
4. **Abstraction:** Complex implementation hide karke clean interface provide karna (`abc.ABC`).

```
from abc import ABC, abstractmethod

# Abstraction: Interface for payment rails
class PaymentRail(ABC):
    @abstractmethod
    def transfer(self, from_acc: str, to_acc: str, amount: float) → bool:
        pass

# Inheritance & Polymorphism
class UpiRail(PaymentRail):
    def transfer(self, from_acc: str, to_acc: str, amount: float) → bool:
        print(f"[UPI] Transferring ₹{amount} from {from_acc} to {to_acc} via NPCI")
        return True

class NeftRail(PaymentRail):
    def transfer(self, from_acc: str, to_acc: str, amount: float) → bool:
        print(f"[NEFT] Queuing ₹{amount} in RBI half-hourly clearing batch")
        return True
```

Magic Methods (Dunder Methods)

Interviewers love asking about `__repr__`, `__eq__`, aur `__lt__` :

```
class BankAccount:
    def __init__(self, account_number: str, balance: float):
        self.account_number = account_number
        self.balance = balance

    # User-friendly string (used by print(acc) and str(acc))
    def __str__(self) → str:
        return f"Account({self.account_number}): ₹{self.balance:,.2f}"

    # Developer/Debug representation (used in interactive shells and logs)
    def __repr__(self) → str:
        return f"BankAccount(account_number='{self.account_number}', balance={self.balance})"

    # Equality check (acc1 == acc2)
    def __eq__(self, other: object) → bool:
        if not isinstance(other, BankAccount):
            return False
        return self.account_number == other.account_number

    # Less-than comparator (Unlocks natural sorting: accounts.sort() by balance!)
    def __lt__(self, other: "BankAccount") → bool:
        return self.balance < other.balance

# Testing Dunder methods:
acc1 = BankAccount("IDFC_101", 45000.0)
acc2 = BankAccount("IDFC_102", 12000.0)
acc3 = BankAccount("IDFC_103", 95000.0)

accounts = [acc1, acc2, acc3]
accounts.sort() # Uses __lt__ automatically!
print(accounts)
# Output sorted by balance: [IDFC_102 (12k), IDFC_101 (45k), IDFC_103 (95k)]
```

4. Python Internals (Memory, GC, Copying & Generators)

1. References & Mutability

📌 KEY ENGINEERING INSIGHT

"Numbers, Strings, Tuples immutable hote hain (unhe modify nahi kiya ja sakta, naya object banta hai). Lists, Dicts, Sets mutable hote hain (same memory address par modify hote hain)."

```
# Pass-by-object-reference trap:
def add_bonus(acc_balance: float, transaction_list: list):
    acc_balance += 500          # Float is immutable → local variable rebinds!
    transaction_list.append(500) # List is mutable → caller's list updates in place!

bal = 1000.0
txns = []
add_bonus(bal, txns)
print(bal)    # Still 1000.0 (Unchanged!)
print(txns)   # [500] (Mutated!)
```

2. Shallow Copy vs Deep Copy

- **Shallow Copy** (`copy.copy()` or `list.copy()`): Outer container ka naya object banta hai, lekin nested objects ke references same rehte hain.
- **Deep Copy** (`copy.deepcopy()`): Recursively saare nested objects ki independent duplicate copy banata hai.

```
import copy

nested = [[1, 2], [3, 4]]
shallow = copy.copy(nested)
deep = copy.deepcopy(nested)

nested[0][0] = 999
print(shallow[0][0]) # 999 (Affected! Shared nested reference)
print(deep[0][0])    # 1   (Unaffected! Fully independent clone)
```

3. Garbage Collection & Memory Management

Python memory management do mechanisms par chalti hai: 1. **Reference Counting (Primary)**: Har object ke header mein `ob_refcnt` hota hai. Jab koi variable us object ko point karta hai, count badhta hai (`+1`). Jab variable scope se bahar nikalta hai ya `del` hota hai, count ghat-ta hai (`-1`). Jaise hi `ob_refcnt == 0`, memory **immediately free** ho jaati hai. 2. **Generational Garbage Collector (Cyclic GC)**: - Agar do objects ek doosre ko refer kar rahe hain (`A.child = B` aur `B.parent = A`), toh dono ka reference count kabhi zero nahi hoga (**Reference Cycle**). - Python ka cyclic GC 3 generations (Gen 0, Gen 1, Gen 2) maintain karta hai. Young objects Gen 0 mein aate hain. GC periodic interval par circular references detect karke collect karta hai.

4. Generators (`yield`) — Processing Millions of Records Without OOM

KEY ENGINEERING INSIGHT

"Agar 10 million transactions ka CSV read karna hai, toh list mein read karne se 8GB RAM bhar jayegi aur server crash ho jayega. Generator ek time par **sirf ek item** memory mein rakhta hai."

```
from typing import Generator

def read_large_transaction_stream(file_path: str) → Generator[dict, None, None]:
    """Generator: Produces one record at a time in O(1) memory."""
    with open(file_path, "r") as f:
        header = f.readline().strip().split(",")
        for line in f:
            values = line.strip().split(",")
            yield dict(zip(header, values)) # Pauses execution and yields control!

# Usage:
# stream = read_large_transaction_stream("huge_txns.csv")
# for txn in stream:
#     process_payment(txn) # Memory stays strictly bounded under 20MB!
```

5. Decorators — Enterprise Cross-Cutting Concerns (Logging, Auth, Timing)

KEY ENGINEERING INSIGHT

"Decorator ek function hota hai jo doosre function ko argument ki tarah leta hai, uski functionality extend karta hai, aur naya function return karta hai."

```
import time
from functools import wraps

def audit_log(func):
    """Decorator that logs execution time and parameters for banking audits."""
    @wraps(func) # Preserves original function's name and docstring
    def wrapper(*args, **kwargs):
        start_time = time.time()
        print(f"[AUDIT] Starting {func.__name__} with args={args}")
        try:
            result = func(*args, **kwargs)
            duration_ms = (time.time() - start_time) * 1000
            print(f"[AUDIT] {func.__name__} completed in {duration_ms:.2f}ms")
            return result
        except Exception as e:
            print(f"[AUDIT ALERT] {func.__name__} failed with error: {e}")
            raise
    return wrapper

@audit_log
def debit_account(account_id: str, amount: float):
    time.sleep(0.05) # Simulate DB latency
    return f"Debited ₹{amount} from {account_id}"

print(debit_account("ACC_1001", 2500.0))
```

6. Context Managers (`with` statement)

KEY ENGINEERING INSIGHT

"Resource cleanup (DB connections close karna, files close karna, locks release karna) guarantee karta hai, chahe exception hi kyu na aa jaye."

```
class DatabaseTransaction:
    """Custom Context Manager simulating ACID transaction scope."""
    def __init__(self, connection_name: str):
        self.conn = connection_name

    def __enter__(self):
        print(f"BEGIN TRANSACTION on {self.conn}")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            print(f"ROLLBACK TRANSACTION due to exception: {exc_val}")
            return False # Re-raise exception
        else:
            print("COMMIT TRANSACTION")
            return True

# Safe usage:
with DatabaseTransaction("IDFC_Core_DB"):
    print("Executing SQL: UPDATE accounts SET balance = balance - 1000")
    # If error happens here, ROLLBACK triggers automatically!
```

5. Advanced Python Tools (`collections` , `itertools` , `dataclasses`)

```
# 1. dataclasses: Clean data containers without boilerplate __init__ and __repr__
from dataclasses import dataclass

@dataclass
class CustomerProfile:
    customer_id: str
    pan_number: str
    kyc_verified: bool
    risk_score: float = 0.0

cust = CustomerProfile("CUST_101", "ABCDE1234F", True)
print(cust) # CustomerProfile(customer_id='CUST_101', pan_number='ABCDE1234F',
kyc_verified=True, risk_score=0.0)

# 2. functools.lru_cache: Built-in Memoization
from functools import lru_cache

@lru_cache(maxsize=128)
def calculate_compound_interest(principal: float, rate: float, years: int) → float:
    print(f"Computing interest for P={principal}, r={rate}, t={years}")
    return principal * ((1 + rate) ** years)

# First call computes; second call returns from cache in O(1)!
calculate_compound_interest(100000, 0.07, 5)
calculate_compound_interest(100000, 0.07, 5) # Instant cache hit!
```

6. Quick Revision & Interview Flashcards

- **Variables = Pointers:** Python variables heap objects ko refer karte hain.
- **Reference Counting + Generational GC:** Python ref count 0 hote hi free karta hai; cycles ko cyclic GC handle karta hai.
- **Generators (`yield`):** Lazy evaluation, memory consumption strictly $O(1)$.
- **Decorators:** Functions that wrap other functions (syntactic sugar `@decorator`).
- **Context Managers (`__enter__` , `__exit__`):** Guarantees resource cleanup even on crashes.

CHAPTER 02 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Python ka memory model (pointers, heap allocations, mutability).
- Generators (`yield`) vs Lists memory footprint.
- Decorators kaise kaam karte hain under the hood.

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 03

NumPy: Numerical Computing & Vectorized Operations

PRIORITY

● High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

1 2 3 4 NumPy: Numerical Computing & Vectorized Operations

1. What is NumPy & Why Does It Exist?

Simple Language Mein:

🔑 30–SECOND INTERVIEW ANSWER (EXECUTIVE PITCH)

"Python ke normal `list` mein agar hum 10 million numbers ka sum karein ya unhe 2 se multiply karein, toh Python ka loop bohot slow chalta hai kyunki Python har element ke liye type checking karta hai aur memory mein pointers follow karta hai.

NumPy (Numerical Python) C-language mein likhi hui library hai jo numerical data ko **contiguous block of memory** mein store karti hai aur CPU ke hardware instructions (**SIMD – Single Instruction Multiple Data**) use karke ek saath hazaron calculations karti hai bina kisi Python loop ke. Isi ko **Vectorization** kehte hain."

2. NumPy ndarray vs Python List

```
flowchart TD
    subgraph PyList["Python List: Array of Pointers (Scattered Heap Memory)"]
        L0["Pointer 0"] --> O0["PyObject: Int 10\n(Header, RefCount, Type, Value)"]
        L1["Pointer 1"] --> O1["PyObject: Int 20\n(Header, RefCount, Type, Value)"]
        L2["Pointer 2"] --> O2["PyObject: Int 30\n(Header, RefCount, Type, Value)"]
    end

    subgraph NumPyArray["NumPy ndarray: Contiguous C-Style Buffer in RAM"]
        N["[ int64: 10 ][ int64: 20 ][ int64: 30 ] (Raw 8-byte integers in sequential memory)"]
    end
```

Feature	Python Standard <code>list</code>	NumPy <code>ndarray</code>
Memory Layout	Pointers to individual heap objects	Contiguous raw memory block
Data Types	Heterogeneous (can mix <code>int</code> , <code>str</code> , <code>dict</code>)	Homogeneous (all elements share exact same <code>dtype</code>)
Performance	Slow (Type checking & pointer dereferencing)	Blazing fast (Compiled C code + CPU SIMD vectorization)
Memory Size	High overhead (~28 bytes per integer)	Minimal (8 bytes for <code>int64</code> , 4 bytes for <code>float32</code>)
Operations	<code>list * 2</code> duplicates list (<code>[1, 2] * 2 = [1, 2, 1, 2]</code>)	<code>arr * 2</code> multiplies every element (<code>[2, 4]</code>)

3. Core Concepts: Dimensions, Shape, Size, Dtype & Axis

```
import numpy as np

# 2D Array: Shape (2, 3) → 2 rows, 3 columns
matrix = np.array([
    [10, 20, 30],
    [40, 50, 60]
], dtype=np.float64)

print("Shape:", matrix.shape) # (2, 3)
print("Dimensions:", matrix.ndim) # 2 (2D array)
print("Total Elements (Size):", matrix.size) # 6
print("Data Type:", matrix.dtype) # float64
```

The "Axis" Concept Demystified:

 **KEY ENGINEERING INSIGHT**

"2D array mein: - `axis=0` hamesha **Rows ke across vertical direction** mein operate karta hai (Column-wise result deta hai). - `axis=1` hamesha **Columns ke across horizontal direction** mein operate karta hai (Row-wise result deta hai)."

```
# Sum along axis=0 (Down the columns):
print(np.sum(matrix, axis=0)) # [10+40, 20+50, 30+60] → [50., 70., 90.]

# Sum along axis=1 (Across the rows):
print(np.sum(matrix, axis=1)) # [10+20+30, 40+50+60] → [60., 150.]
```

4. Creating NumPy Arrays

```
# 1. Direct from list
a = np.array([1, 2, 3])

# 2. Zeros and Ones (Allocation without garbage data)
zeros = np.zeros((3, 4)) # 3 rows, 4 columns of 0.0
ones = np.ones((2, 3), dtype=np.int32) # 2x3 matrix of 1s

# 3. arange: Like range() but returns ndarray (start, stop, step)
seq = np.arange(10, 50, 5) # [10, 15, 20, 25, 30, 35, 40, 45]

# 4. linspace: Linear spacing (start, stop, num_points)
# Great for financial curves and probability distributions
curve = np.linspace(0.0, 1.0, 5) # [0.0, 0.25, 0.5, 0.75, 1.0]

# 5. Identity Matrix: Diagonal 1s, rest 0s
eye_matrix = np.eye(3) # 3x3 identity matrix
```

5. Array Operations: Reshaping, Slicing & Views vs Copies

Slicing & The "View" Trap:

KEY ENGINEERING INSIGHT

"NumPy mein slice lene par data copy nahi hota; woh original array ka **View (Window)** hota hai. Agar view modify karoge, toh original array bhi change ho jayega!"

```
arr = np.array([10, 20, 30, 40, 50])
view_slice = arr[1:4] # Elements: [20, 30, 40]
view_slice[0] = 999 # Modifying the view!
print(arr) # [10, 999, 30, 40, 50] → ORIGINAL ARRAY MUTATED!

# Explicit Copy (Safe):
safe_copy = arr[1:4].copy()
safe_copy[0] = 777
print(arr[1]) # 999 (Original remains untouched)
```

Reshaping & Flattening:

```
orig = np.arange(12) # [0, 1, 2, ... 11]

# Reshape to 3x4 matrix
reshaped = orig.reshape(3, 4)

# Reshape with -1 (NumPy automatically calculates the remaining dimension!)
auto_reshaped = orig.reshape(2, -1) # Shape becomes (2, 6)

# Flattening: flatten() vs ravel()
# flatten() hamesha naya copy banata hai
flat_copy = reshaped.flatten()

# ravel() view return karne ki koshish karta hai (zero memory copy!)
flat_view = reshaped.ravel()
```

6. Vectorization: Why It Beats Python Loops by 50x

Code Benchmark: Python Loop vs NumPy Vectorization

```
import time

size = 5_000_000
py_list = list(range(size))
np_arr = np.arange(size)

# Test 1: Python Loop
start = time.time()
py_result = [x * 2 for x in py_list]
py_time = time.time() - start

# Test 2: NumPy Vectorization
start = time.time()
np_result = np_arr * 2
np_time = time.time() - start

print(f"Python List Loop: {py_time:.4f} seconds")
print(f"NumPy Vectorized: {np_time:.4f} seconds")
print(f"Speedup: {py_time / np_time:.1f}x FASTER!")
# Output: NumPy is typically 40x to 80x faster!
```

Kyu fast hai? 1. C-level loop runs inside CPU registers without Python bytecode overhead. 2. Hardware prefetching: Contiguous bytes sequentially load into L1/L2 cache without cache misses. 3. SIMD (AVX-512): Single CPU instruction multiplies 8 floating-point numbers in a single clock cycle.

7. Broadcasting Rules (The Most Asked NumPy Concept)

Simple Language Mein:

KEY ENGINEERING INSIGHT

"Broadcasting ek aisa rule hai jiske through NumPy alag-alag shapes ke arrays par arithmetic operations perform karne deta hai bina memory mein extra copy banaye."

```
flowchart LR
    subgraph Operation["Broadcasting: (3, 3) + (1, 3)"]
        A["Matrix A: (3, 3)\n[[1, 2, 3],\n [4, 5, 6],\n [7, 8, 9]]"]
        B["Row B: (1, 3)\n[[10, 20, 30]]"]
        Result["Row B is conceptually stretched across 3 rows:\n[[11, 22, 33],\n [14, 25, 36],\n [17, 28, 39]]"]
    end
    A --> Result
    B --> Result
```

The Two Strict Rules of Broadcasting:

Piche se (from trailing dimensions to leading dimensions) compare karo: 1. Dono dimensions **equal** honi chahiye, **OR** 2. Kisi ek dimension ka size **1** hona chahiye. Agar dimension 1 hai, toh woh doosre array ke dimension se match karne ke liye virtual stretch (replicate) ho jata hai.

```
# Example: Adding a scalar (Shape: ()) to a 2D matrix (Shape: (2, 2))
mat = np.array([[1, 2], [3, 4]])
result = mat + 10 # 10 broadcasts to [[10, 10], [10, 10]]
# Result: [[11, 12], [13, 14]]
```

8. Boolean Indexing (Filtering Data)

```
# Banking Transaction Amounts:
transactions = np.array([250.0, 12000.0, 450.0, 95000.0, 3200.0, 78000.0])

# Step 1: Boolean mask create karo
mask = transactions > 50000.0
print(mask) # [False, False, False, True, False, True]

# Step 2: Mask se filter karo
suspicious_high_txns = transactions[mask]
print(suspicious_high_txns) # [95000., 78000.]

# Compound filtering (& for AND, | for OR, ~ for NOT):
flagged = transactions[(transactions >= 1000) & (transactions <= 80000)]
```

9. NumPy Interview Question Bank (40+ Questions)

Part A: 20 Conceptual Questions

1. NumPy array aur Python list mein 3 fundamental differences kya hain?

Answer: Contiguous vs pointer memory, homogeneous vs heterogeneous types, element-wise vectorized arithmetic vs repetition.

2. NumPy ka `ndarray` internal structure kaisa hota hai?

Answer: Ek data pointer (raw memory), `dtype`, `shape` tuple, aur `strides` tuple (bytes to step to reach next element along each dimension).

3. What is a Strides tuple in NumPy?

Answer: Har dimension mein agle element tak jump karne ke liye kitne bytes aage badhna hai.
Example: Shape `(2, 3)` ke `int64` array ke strides `(24, 8)` hote hain.

4. Difference between `np.copy()` and View?

5. What is Broadcasting and what are its two mathematical conditions?

6. What is Vectorization and why does it avoid the Global Interpreter Lock (GIL)?

7. Explain the difference between `axis=0` and `axis=1` in a 2D matrix.

8. What does `reshape(3, -1)` do?

9. Difference between `np.zeros()` and `np.empty()` ?

Answer: `zeros` memory ko 0 se initialize karta hai; `empty` uninitialized garbage RAM points return karta hai (slightly faster, but dangerous).

10. Difference between `arr.flatten()` and `arr.ravel()` ?

11. How does boolean indexing work under the hood?

12. Can a NumPy array contain multiple data types? What happens if you pass `[1, "IDFC", 3.5]` ?

Answer: Automatic type upcasting happens. Saare elements string (`<U21`) ban jayenge.

13. What is SIMD and how does NumPy utilize it?

14. How do you find the memory footprint of an array? (`arr.nbytes` or `arr.size * arr.itemsize`).

15. What is the difference between `np.dot()` and `*` operator on two 2D arrays?

Answer: `*` element-wise multiplication karta hai; `np.dot()` / `@` linear algebra matrix multiplication karta hai.

16. How does `np.nan` behave in comparisons (`np.nan == np.nan`)?

Answer: `False` ! Always use `np.isnan(val)`.

17. What is the purpose of `np.linspace()` versus `np.arange()` ?

18. What is Fancy Indexing in NumPy?

19. How do you replace values in an array conditioned on a boolean predicate? (`np.where(condition, x, y)`).

20. Why is `np.sum(arr)` faster than Python's built-in `sum(arr)` on a NumPy array?

Part B: 10 Coding Questions

- 1. Normalize Array:** Given a 1D array of account balances, scale them between 0 and 1: `(arr - arr.min()) / (arr.max() - arr.min())`.
- 2. Find Indices of Outliers:** Return indices of transactions that are 3 standard deviations away from the mean: `np.where(np.abs(arr - np.mean(arr)) > 3 * np.std(arr))[0]`.
- 3. Count Non-Zero Elements:** `np.count_nonzero(arr)`.
- 4. Extract Diagonal Elements:** `np.diag(matrix)`.
- 5. Clip Values (Upper and Lower bounds):** Restrict transaction amounts between ₹100 and ₹50,000: `np.clip(arr, 100, 50000)`.
- 6. Reverse Array In-Place:** `arr[::-1]` (view) or `np.flip(arr)`.
- 7. Find Most Frequent Value in Array:** `vals, counts = np.unique(arr, return_counts=True); vals[np.argmax(counts)]`.
- 8. Replace all NaN with Mean:** `arr[np.isnan(arr)] = np.nanmean(arr)`.
- 9. Matrix Multiplication:** `C = A @ B` or `np.matmul(A, B)`.
- 10. Compute Moving / Rolling Average using convolution:** `np.convolve(arr, np.ones(window)/window, mode='valid')`.

Part C: 10 Comparison Questions

1. `list.append()` vs `np.append()` ? (Note: `np.append` creates a brand new array every time; never use it in a loop!).
2. `np.concatenate` vs `np.vstack` vs `np.hstack` ?
3. In-place operation (`arr += 1`) vs new assignment (`arr = arr + 1`)?
4. `np.mean()` vs `np.nanmean()` ?
5. `np.argmax()` vs `np.max()` ?
6. Deep copy vs shallow view in multi-dimensional slicing?
7. Integer array indexing vs boolean mask indexing?
8. Memory footprint: 1,000,000 Python ints vs 1,000,000 `np.int32` ? (~28MB vs 4MB).
9. Memory order: C-contiguous (row-major) vs Fortran-contiguous (column-major)?
10. `np.ascontiguousarray()` kab use karte hain?

CHAPTER 03 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Core fundamentals and architectural mechanics of NumPy: Numerical Computing & Vectorized Operations
- Production banking scenarios and transaction reliability
- High-frequency technical interview questions and follow-ups

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 04

pandas: Tabular Data Analysis & Financial DataFrames

PRIORITY

● High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)



pandas: Tabular Data Analysis & Financial DataFrames

1. What is pandas & Why Does It Exist?

Simple Language Mein:



BANKING & FINTECH ARCHITECTURE SCENARIO

"Agar NumPy ek n-dimensional mathematical array hai, toh **pandas** Python ka Excel / SQL Table hai.

Real-world banking data messy hota hai: usme column names hote hain (`account_id` , `amount`), row labels hote hain, missing values hoti hain (`NULL` / `NaN`), aur date-times hote hain.

pandas tabular data (rows and columns) ko manipulate karne, clean karne, aggregate karne (`groupby`), aur SQL-style join karne ke liye de-facto industry standard library hai."

2. Core Data Structures: Series vs DataFrame

```
flowchart TD
    subgraph DataStructures["pandas Core Architecture"]
        S["Series (1D Array)\nSingle column with explicit Index\nExample: All transaction amounts"]
        DF["DataFrame (2D Table)\nCollection of Series sharing the same Index\nExample: Full transactions table (id, amount, date)"]
        S --> DF
    end
```


1. Creating DataFrames & Series

```
import pandas as pd
import numpy as np

# 1. pd.Series: 1D labeled array
balances = pd.Series([45000, 12000, 95000], index=["Aarav", "Priya", "Rohan"], name="balance")
print(balances["Priya"]) # 12000

# 2. pd.DataFrame: 2D labeled table
data = {
    "account_id": [101, 102, 103],
    "customer": ["Aarav", "Priya", "Rohan"],
    "balance": [45000.0, 12000.0, 95000.0],
    "account_type": ["SAVINGS", "CURRENT", "SAVINGS"]
}
df = pd.DataFrame(data)
```

3. Loading Data From Different Sources

```
# CSV
df_csv = pd.read_csv("01-PYTHON/transactions.csv")

# JSON
df_json = pd.read_json("accounts.json")

# SQL Database (Direct connection via SQLAlchemy/sqlite3)
import sqlite3
conn = sqlite3.connect("banking.db")
df_sql = pd.read_sql_query("SELECT * FROM accounts WHERE status = 'ACTIVE'", conn)
conn.close()
```

3. Inspecting & Summarizing Data (First 60 Seconds in EDA)

```
# Load sample dataset
df = pd.read_csv("01-PYTHON/transactions.csv")

# 1. Quick preview:
print(df.head(3)) # First 3 rows
print(df.tail(2)) # Last 2 rows

# 2. Dimensions & Memory:
print("Shape (Rows, Cols):", df.shape) # (1000, 8)
print("Columns:", df.columns.tolist())

# 3. Structural Health Check (Missing values + Dtypes):
df.info() # Displays non-null counts and memory usage

# 4. Statistical Summary (Mean, Min, 25%, 50%, 75%, Max for numerical columns):
print(df.describe())
```

4. Selecting Data: `loc` vs `iloc` (The Top Interview Trap!)

KEY ENGINEERING INSIGHT

[!IMPORTANT]

Flashcard: `loc` vs `iloc`

- `df.loc[]` $\implies$ **Label-based** selection (Row name / index label aur column name ke basis par select karta hai). End boundary is **inclusive**!
- `df.iloc[]` $\implies$ **Integer-position based** selection (0-indexed integer position ke basis par select karta hai). End boundary is **exclusive**!

```
# Selecting single column (Returns Series)
amounts = df["amount"]

# Selecting multiple columns (Returns DataFrame)
subset = df[["transaction_id", "amount", "status"]]

# loc vs iloc:
# Let's say index is 0, 1, 2...
# loc[row_label, col_label]
print(df.loc[0:2, ["customer_id", "amount"]]) # Returns rows labeled 0, 1, 2 (INCLUSIVE!)

# iloc[row_pos, col_pos]
print(df.iloc[0:2, 0:3]) # Returns rows at position 0, 1 (2 is EXCLUDED!)
```

5. Filtering Data (Boolean Masking)

```
# 1. Single condition: High-value transactions
high_val = df[df["amount"] > 50000]

# 2. Multiple conditions: Use & (AND), | (OR), ~ (NOT) with parentheses ()!
# WARNING: 'and' / 'or' will throw a ValueError in pandas!
flagged_txns = df[
    (df["amount"] > 25000) &
    (df["status"] == "FAILED") &
    (df["merchant_category"] != "UTILITIES")
]

# 3. isin(): SQL 'IN' equivalent
metro_merchants = df[df["merchant_category"].isin(["FOOD", "TRAVEL"])]

# 4. query(): Readable string expressions
filtered_via_query = df.query("amount > 50000 and status == 'SUCCESS'")
```

6. Handling Missing Data (NaN / None)

```
# 1. Detect missing values
print(df.isna().sum()) # Count of NaNs per column

# 2. Dropping missing data
# dropna(axis=0) drops rows; dropna(axis=1) drops columns
clean_df = df.dropna(subset=["amount"]) # Drop rows where amount is missing

# 3. Imputation strategies (Filling missing data):
# A. Fill with static constant:
df["merchant_category"] = df["merchant_category"].fillna("UNKNOWN")

# B. Impute with median/mean (Prevents skewing):
median_amount = df["amount"].median()
df["amount"] = df["amount"].fillna(median_amount)

# C. Forward Fill (ffill) & Backward Fill (bfill) - Great for stock prices & time-series!
# df["price"] = df["price"].ffill() # Takes previous known value
```

7. Data Cleaning & Transformations

```
# 1. Removing Duplicates
df = df.drop_duplicates(subset=["transaction_id"], keep="first")

# 2. Type Casting (astype)
# Converting float IDs to integers or category types
df["account_id"] = df["account_id"].astype(str)

# 3. String Operations (.str accessor)
df["customer_id_clean"] = df["customer_id"].str.strip().str.upper()

# 4. DateTime Parsing
df["timestamp"] = pd.to_datetime(df["timestamp"])
df["year"] = df["timestamp"].dt.year
df["month"] = df["timestamp"].dt.month
df["day_name"] = df["timestamp"].dt.day_name()
df["hour"] = df["timestamp"].dt.hour

# 5. Renaming Columns
df = df.rename(columns={"timestamp": "txn_timestamp", "amount": "txn_amount"})
```

8. GroupBy: Split \$\to\$ Apply \$\to\$ Combine (HIGH PRIORITY)

```
flowchart TD
    subgraph GroupByPipeline["The Split-Apply-Combine Workflow"]
        T["Input DataFrame (1,000 Rows)"] --> S["1. SPLIT by 'merchant_category' into distinct buckets"]
        S --> A["2. APPLY aggregate functions: sum(), count(), mean()"]
        A --> C["3. COMBINE results into a clean summary DataFrame"]
    end
```

```
# 1. Total and average amount spent per merchant category
category_summary = df.groupby("merchant_category")["txn_amount"].agg(
    total_volume="sum",
    avg_ticket_size="mean",
    txn_count="count"
).reset_index()

print(category_summary.sort_values(by="total_volume", ascending=False))

# 2. Multi-Level GroupBy: Group by Customer and Transaction Status
cust_status = df.groupby(["customer_id", "status"])["txn_amount"].sum().unstack(fill_value=0)
# Displays customers as rows, status ('FAILED', 'SUCCESS') as columns!
```

9. Merging & Joining (SQL Joins in pandas)

```
# Sample customer KYC DataFrame
customers_df = pd.DataFrame({
    "customer_id": ["CUST_101", "CUST_102", "CUST_103"],
    "kyc_status": ["VERIFIED", "VERIFIED", "PENDING"],
    "city": ["Bengaluru", "Mumbai", "Delhi"]
})

# 1. Inner Merge (SQL: INNER JOIN)
merged_inner = pd.merge(df, customers_df, on="customer_id", how="inner")

# 2. Left Merge (SQL: LEFT OUTER JOIN)
# Preserves all transactions even if customer KYC record is missing
merged_left = pd.merge(df, customers_df, on="customer_id", how="left")

# 3. Concatenation (SQL: UNION ALL)
# Binding two DataFrames vertically
# df_all = pd.concat([df_jan, df_feb], axis=0, ignore_index=True)
```

10. Pivot Tables

```
# Pivot Table: Cross-tabulating Merchant Category vs Transaction Type
pivot = pd.pivot_table(
    df,
    values="txn_amount",
    index="merchant_category",
    columns="transaction_type",
    aggfunc=["sum", "count"],
    fill_value=0,
    margins=True # Adds 'All' Grand Total row & column!
)
print(pivot)
```

11. `apply()` vs `map()` vs Vectorization

```
# 1. map(): Works on Series for element-wise replacement via dict
status_codes = {"SUCCESS": 1, "FAILED": 0, "PENDING": 2}
df["status_code"] = df["status"].map(status_codes)

# 2. apply(): Runs custom Python function row-by-row or col-by-col
# NOTE: Slower than vectorization because it invokes Python function N times!
def categorize_risk(amount):
    if amount > 50000:
        return "HIGH_RISK"
    elif amount > 10000:
        return "MEDIUM_RISK"
    return "LOW_RISK"

df["risk_tier"] = df["txn_amount"].apply(categorize_risk)

# 3. Vectorized Alternative using np.select (100x FASTER than .apply!):
conditions = [
    df["txn_amount"] > 50000,
    df["txn_amount"] > 10000
]
choices = ["HIGH_RISK", "MEDIUM_RISK"]
df["risk_tier_vectorized"] = np.select(conditions, choices, default="LOW_RISK")
```

12. Performance Optimization for Large Banking Datasets

1. **Avoid `for index, row in df.iterrows():`** Iterating rows manually is the single slowest anti-pattern in pandas. Always use vectorized operations or list comprehensions.
2. **Use Categorical Dtypes (`astype('category')`)**: If a column has few unique strings (e.g., `status` has only 3 values: `SUCCESS`, `FAILED`, `PENDING`), converting to `category` saves up to **80% RAM**.
`python df["status"] = df["status"].astype("category")`
3. **Chunking Huge CSV Files (Processing 10GB in 500MB RAM)**: `python chunk_size = 50000`
`total_volume = 0 for chunk in pd.read_csv("10gb_transactions.csv", chunksize=chunk_size):`
`total_volume += chunk["amount"].sum()`

13. pandas Interview Question Bank (40 Questions)

1. **Difference between `loc` and `iloc`**? (Label-based inclusive vs integer-position exclusive).
2. **What is the difference between a pandas Series and a 1D NumPy array?** (Series has an explicit index, handles heterogeneous missing types, and provides alignment).
3. **How does pandas represent missing data in numerical columns vs string columns?** (`np.nan` / `float64` historically, `pd.NA` in nullable dtypes).
4. **Explain the Split-Apply-Combine paradigm in `groupby()`**.
5. **Difference between `df.dropna()` and `df.fillna()`**?
6. **What is the difference between `merge()` and `concat()`**?
7. **How do you perform a SQL `LEFT JOIN` in pandas?** (`pd.merge(df1, df2, on='key', how='left')`).

8. Why is `apply()` generally slower than vectorized operations? (Runs Python bytecode per row, losing C-level SIMD benefits).
9. How do you optimize memory usage of an object/string column? (Convert to `category` dtype).
10. How do you read a 20GB CSV file in pandas on a laptop with 8GB RAM? (Using `chunksize` parameter in `pd.read_csv`).
11. Difference between `sort_values()` and `sort_index()` ?
12. How does `df.duplicated()` work and how do you keep the last duplicate? (`keep='last'`).
13. What does `reset_index()` do after a `groupby` aggregation? (Converts grouping keys from index back into regular columns).
14. How do you filter rows based on multiple conditions? Why can't we use `and` / `or` ? (Must use bitwise `&` and `|` with parentheses because Python's boolean short-circuiting evaluates the whole Series as truthy/falsy).
15. What is the difference between `df.isnull()` and `df.isna()` ? (They are identical aliases in pandas).
16. How do you convert a string column to datetime? (`pd.to_datetime(col, format=...)`).
17. How do you calculate a rolling 7-day average of transactions?
(`df['amount'].rolling(window=7).mean()`).
18. Difference between `unstack()` and `stack()` ?
19. What is a MultiIndex in pandas?
20. How do you export a cleaned DataFrame directly to a SQL database table?
(`df.to_sql(name='clean_txns', con=engine, if_exists='append', index=False)`).
21. How do you rename specific columns in a DataFrame? (`df.rename(columns={'old': 'new'})`).
22. What is `value_counts()` and what does `normalize=True` do? (Returns frequency counts; `normalize=True` returns relative proportions/percentages).
23. Difference between `transform()` and `apply()` in `groupby` ? (`transform()` returns a Series with the same shape as the original DataFrame, ideal for calculating customer percentage contributions; `apply()` aggregates/reduces dimensions).
24. How do you find the percentage of missing values in each column? (`df.isna().mean() * 100`).
25. How do you replace outliers in a DataFrame? (`np.clip()` or condition masking).
26. What is `melt()` in pandas? (Opposite of pivot: unpivots a DataFrame from wide format to long format).
27. Difference between `Series.map()` and `Series.apply()` ?
28. How do you select rows where a string column contains a substring?
(`df['col'].str.contains('IDFC', case=False, na=False)`).
29. What does `inplace=True` do and why is it deprecated / discouraged in modern pandas? (Doesn't guarantee zero copy under the hood; method chaining is cleaner).
30. How do you handle timezone-aware datetime columns in pandas? (`.dt.tz_localize()` and `.dt.tz_convert('Asia/Kolkata')`).

31. How do you find the top 5 transactions by amount per customer?

```
( df.groupby('cust_id').apply(lambda x: x.nlargest(5, 'amount')) ).
```

32. What is `crosstab()` ? (Cross-tabulation of two factor columns, specialized frequency pivot).

33. Difference between `copy(deep=True)` and `copy(deep=False)` in pandas?

34. How do you set a custom index in a DataFrame? (`df.set_index('account_id')`).

35. What does `df.select_dtypes(include=[np.number])` do? (Selects only numeric columns).

36. How do you merge two DataFrames on different column names? (`pd.merge(df1, df2, left_on='from_acc', right_on='acc_id')`).

37. What is the difference between `cumsum()` and `cumprod()` ?

38. How do you sample 10% of rows randomly from a large DataFrame? (`df.sample(frac=0.10, random_state=42)`).

39. How do you compute correlation between columns? (`df.corr()`).

40. How does pandas handle categorical encoding for machine learning or reporting?

```
( pd.get_dummies(df, columns=['category']) ).
```

CHAPTER 04 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- Core fundamentals and architectural mechanics of pandas: Tabular Data Analysis & Financial DataFrames
- Production banking scenarios and transaction reliability
- High-frequency technical interview questions and follow-ups

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 05

Matplotlib: Data Visualization & Financial Plotting

PRIORITY

● High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)



Matplotlib: Data Visualization & Financial Plotting

1. What is Matplotlib & Why Does It Exist?

Simple Language Mein:



KEY ENGINEERING INSIGHT

"Data ko tables ya numbers mein dekh kar human brain patterns quickly catch nahi kar pata. Agar fraud transactions spike ho rahe hain ya monthly card spending badh rahi hai, toh **Visual Charts** sabse tezi se insights dete hain.

Matplotlib Python ki foundational 2D plotting library hai. pandas ke internal visualization methods (`df.plot()`) aur Seaborn sabhi under the hood Matplotlib par hi chalte hain."

2. The Anatomy of a Figure: Figure vs Axes vs Axis

```
flowchart TD
    subgraph FigureCanvas["Figure (The Complete Canvas / Window)"]
        subgraph Ax1["Axes 1 (Plot Sub-window 1)"]
            T1["Title & Legend"]
            Plot1["Line Plot / Bar Chart"]
            XAxis1["X-Axis (Ticks, Labels)"]
            YAxis1["Y-Axis (Ticks, Labels)"]
        end
        subgraph Ax2["Axes 2 (Plot Sub-window 2)"]
            Plot2["Histogram / Scatter"]
        end
    end
    end
```


30-SECOND INTERVIEW ANSWER (EXECUTIVE PITCH)

[!IMPORTANT]

Flashcard: Figure vs Axes

- **Figure:** Complete image ya canvas (page size, background).
- **Axes:** Actual graph ya plotting area (jisme x-axis, y-axis, lines, bars, title hote hain). Ek Figure mein multiple Axes (subplots) ho sakte hain!
- **Axis:** Specific number line (X-axis horizontal ya Y-axis vertical jo ticks aur limits define karti hai).

3. Two Interfaces: Pyplot (Stateful) vs Object-Oriented API

Approach A: Pyplot Interface (`plt.plot`)

Matlab-style global state machine. Simple scripts ke liye theek hai, lekin complex subplots mein confusing hota hai:

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [10, 20, 25])
plt.title("Simple Line")
# plt.show()
```

Approach B: Object-Oriented API (`fig, ax = plt.subplots()`) — RECOMMENDED!

Explicitly Figure aur Axes objects banate hain. Production dashboards aur multi-panel charts ke liye industry standard hai:

```
fig, ax = plt.subplots(figsize=(8, 4)) # Width: 8 inches, Height: 4 inches
ax.plot([1, 2, 3], [10, 20, 25], color="darkred", marker="o", label="Daily Spend")
ax.set_title("IDFC Customer Spend Trend", fontsize=14, fontweight="bold")
ax.set_xlabel("Days")
ax.set_ylabel("Amount (₹)")
ax.legend()
ax.grid(True, linestyle="--", alpha=0.6)
# fig.savefig("spend_trend.png")
```

4. The 5 Essential Chart Types in Banking

```
import matplotlib.pyplot as plt
import numpy as np

# Sample Data
categories = ["Food", "Shopping", "Travel", "Utilities", "Healthcare"]
spend = [18500, 42000, 12500, 8400, 15000]
months = ["Jan", "Feb", "Mar", "Apr", "May"]
monthly_inflow = [50000, 52000, 48000, 65000, 70000]
tx_amounts = np.random.exponential(scale=2500, size=500)

# Create 2x2 multi-panel figure
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# 1. Line Chart: Time-Series Trend
axes[0, 0].plot(months, monthly_inflow, marker="s", color="#9b1c1c", linewidth=2)
axes[0, 0].set_title("Monthly Salary Inflow Trend")
axes[0, 0].grid(True)

# 2. Bar Chart: Categorical Comparison
axes[0, 1].bar(categories, spend, color="#2b6cb0")
axes[0, 1].set_title("Spend by Merchant Category")
axes[0, 1].tick_params(axis="x", rotation=30)

# 3. Histogram: Distribution of Transaction Amounts
axes[1, 0].hist(tx_amounts, bins=25, color="#38a169", edgecolor="black")
axes[1, 0].set_title("Transaction Value Distribution")
axes[1, 0].set_xlabel("Amount (₹)")

# 4. Scatter Plot: Transaction Amount vs Fraud Risk Score
risk_scores = np.random.uniform(0, 100, 500)
axes[1, 1].scatter(tx_amounts, risk_scores, alpha=0.5, color="#d69e2e")
axes[1, 1].set_title("Amount vs Anomaly Risk Score")
axes[1, 1].set_xlabel("Amount")
axes[1, 1].set_ylabel("Risk (0-100)")

plt.tight_layout() # Prevents label overlapping!
# plt.savefig("dashboard.png")
```

5. Matplotlib Interview Question Bank (20 Questions)

1. Figure aur Axes mein kya difference hai?

Answer: Figure pura outer canvas/window hota hai; Axes actual plot area hota hai jisme data points, ticks, aur labels render hote hain.

2. Why is the Object-Oriented API (`fig, ax = plt.subplots()`) preferred over `plt.plot()` ?

Answer: OO API explicit references deta hai, multiple subplots ko cleanly manage karta hai, aur global state bugs avoid karta hai.

3. What is the purpose of `plt.tight_layout()` ?

Answer: Automatically adjusts subplots padding to prevent overlapping titles and axis labels.

4. How do you save a chart as an image without showing a GUI popup window?

Answer: `fig.savefig('chart.png', dpi=300, bbox_inches='tight')`.

5. How do you run Matplotlib on a headless Linux server without a display/monitor?

Answer: Use non-interactive backend before importing pyplot: `python import matplotlib`
`matplotlib.use('Agg') import matplotlib.pyplot as plt`

6. When should you use a Histogram vs a Bar Chart?

Answer: Bar chart categorical discrete data compare karta hai (e.g., spending by category); Histogram continuous numerical data ki frequency distribution dikhata hai (e.g., frequency of amounts between ₹100 and ₹1,000).

7. What is the difference between a Scatter plot and a Line plot?

8. How do you add annotations or callout text to highlight an outlier on a plot?

Answer: `ax.annotate('Suspicious Spike!', xy=(x, y), xytext=(x+1, y+500), arrowprops=dict(facecolor='red'))`.

9. How do you plot twin axes (e.g., primary Y-axis for Transaction Volume and secondary Y-axis for Fraud Percentage)?

Answer: `ax2 = ax1.twinx()`.

10. What parameter controls marker transparency in scatter plots to handle overplotting?

Answer: `alpha` parameter (e.g., `alpha=0.3`).

11. How do you customize X-axis date formatting in time-series plots?

Answer: `import matplotlib.dates as mdates;`
`ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))`.

12. How do you rotate tick labels so they don't overlap?

Answer: `ax.tick_params(axis='x', rotation=45)` or `plt.xticks(rotation=45)`.

13. How do you plot a horizontal bar chart?

Answer: `ax.barh(categories, values)`.

14. How do you change the color palette or style sheet in Matplotlib?

Answer: `plt.style.use('seaborn-v0_8-whitegrid')` or `plt.style.use('ggplot')`.

15. What is a Box Plot (`ax.boxplot()`) and what 5 summary metrics does it display?

Answer: Minimum, 25th percentile (Q1), Median, 75th percentile (Q3), Maximum, and Outliers beyond $1.5 \times \text{IQR}$.

16. How do you add a horizontal benchmark line across a chart?

Answer: `ax.axhline(y=50000, color='red', linestyle='--')`.

17. How do you control the resolution of saved image files?

Answer: The `dpi` parameter in `savefig()` (e.g., `dpi=300` for publication/print quality).

18. How do you close figures to prevent memory leaks in long-running services?

Answer: `plt.close(fig)` or `plt.close('all')`. If you don't close figures in a loop, memory keeps accumulating in the GUI backend.

19. How do you create a pie chart with percentage labels displayed?

Answer: `ax.pie(values, labels=labels, autopct='%1.1f%%')`.

20. How does pandas integrate with Matplotlib?

Answer: `df.plot(kind='bar', ax=ax)` directly attaches pandas plot rendering onto existing Matplotlib Axes objects.

 CHAPTER 05 SUMMARY & REVISION CHECKLIST **WHAT TO REMEMBER**

- Core fundamentals and architectural mechanics of Matplotlib: Data Visualization & Financial Plotting
- Production banking scenarios and transaction reliability
- High-frequency technical interview questions and follow-ups

 CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 06

SQL pandas Rosetta Stone: Translating SQL Knowledge to pandas

PRIORITY

● High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)



SQL ↔ pandas Rosetta Stone: Translating SQL Knowledge to pandas

1. Why This Matters

Aapka SQL background already solid hai. Nayi syntax zero se yaad karne ke bajaye, agar aap **SQL queries ko direct pandas operations se map** kar loge, toh aap 1 ghante ke andar pandas mein expert level analysis likhne lagoge!

2. The Core Conceptual Mapping

```
flowchart LR
    subgraph RelationalSQL["Relational Database (SQL)"]
        Table["Table"]
        Row["Row / Record"]
        Column["Column / Field"]
        PrimaryKey["Primary Key"]
    end

    subgraph PythonPandas["pandas Ecosystem"]
        DF["DataFrame"]
        SeriesRow["Index Entry"]
        SeriesCol["Series"]
        Index["DataFrame Index"]
    end

    Table <--> DF
    Row <--> SeriesRow
    Column <--> SeriesCol
    PrimaryKey <--> Index
```

3. Clause-by-Clause Translation Table

1. **SELECT** (Column Projection)

SQL

```
SELECT transaction_id, amount
FROM transactions;

SELECT DISTINCT merchant_category
FROM transactions;
```

pandas

```
# Select columns
df[["transaction_id", "amount"]]

# DISTINCT
df["merchant_category"].unique() # or df[["merchant_category"]].drop_duplicates()
```

2. **WHERE** (Filtering & Predicates)

SQL

```
SELECT * FROM transactions
WHERE amount > 10000 AND status = 'SUCCESS';

SELECT * FROM transactions
WHERE merchant_category IN ('FOOD', 'TRAVEL');

SELECT * FROM transactions
WHERE merchant_category IS NULL;
```

pandas

```
# Multiple conditions (& for AND, | for OR, ~ for NOT)
df[(df["amount"] > 10000) & (df["status"] == "SUCCESS")]

# IN condition (isin)
df[df["merchant_category"].isin(["FOOD", "TRAVEL"])]

# IS NULL / IS NOT NULL
df[df["merchant_category"].isna()]
df[df["merchant_category"].notna()]
```

3. **ORDER BY** & **LIMIT** (Sorting & Pagination)

SQL

```
SELECT * FROM transactions
ORDER BY amount DESC, timestamp ASC
LIMIT 10 OFFSET 20;
```

pandas

```
# sort_values()
df.sort_values(
    by=["amount", "timestamp"],
    ascending=[False, True]
).iloc[20:30] # Limit 10 Offset 20
```

4. GROUP BY & HAVING (Aggregations)

SQL

```
SELECT
    merchant_category,
    COUNT(*) AS total_txns,
    SUM(amount) AS total_spend,
    AVG(amount) AS avg_spend
FROM transactions
GROUP BY merchant_category
HAVING SUM(amount) > 100000;
```

pandas

```
# groupby() + agg()
category_summary = df.groupby("merchant_category").agg(
    total_txns=("amount", "count"),
    total_spend=("amount", "sum"),
    avg_spend=("amount", "mean")
).reset_index()

# HAVING clause filter:
category_summary[category_summary["total_spend"] > 100000]
```

5. JOIN (Relational Merging)

SQL

```
-- INNER JOIN
SELECT t.*, c.first_name, c.phone
FROM transactions t
INNER JOIN customers c ON t.customer_id = c.customer_id;

-- LEFT OUTER JOIN
SELECT t.*, c.first_name
FROM transactions t
LEFT JOIN customers c ON t.customer_id = c.customer_id;
```

pandas

```
# INNER JOIN
pd.merge(transactions_df, customers_df, on="customer_id", how="inner")

# LEFT JOIN
pd.merge(transactions_df, customers_df, on="customer_id", how="left")

# RIGHT JOIN: how='right'
# FULL OUTER JOIN: how='outer'
```

6. UNION ALL & UNION (Vertical Concatenation)

SQL

```
SELECT * FROM transactions_jan
UNION ALL
SELECT * FROM transactions_feb;
```

pandas

```
# UNION ALL
pd.concat([jan_df, feb_df], axis=0, ignore_index=True)

# UNION (with deduplication)
pd.concat([jan_df, feb_df], axis=0, ignore_index=True).drop_duplicates()
```

7. Window Functions (OVER (PARTITION BY ... ORDER BY ...))

SQL Running Balance

```
SELECT
    transaction_id,
    account_id,
    amount,
    SUM(amount) OVER (
        PARTITION BY account_id
        ORDER BY timestamp
    ) AS running_balance
FROM transactions;
```

pandas Equivalent

```
# Groupby + cumsum()
df["running_balance"] = df.sort_values("timestamp").groupby("account_id")["amount"].cumsum()
```


SQL Ranking (`ROW_NUMBER()` / `DENSE_RANK()`)

```
SELECT
  *,
  DENSE_RANK() OVER (
    PARTITION BY customer_id
    ORDER BY amount DESC
  ) AS rank_in_customer
FROM transactions;
```

pandas Equivalent

```
# rank(method='dense')
df["rank_in_customer"] = df.groupby("customer_id")["amount"].rank(
    method="dense",
    ascending=False
)
```

SQL `LAG()` / `LEAD()` (Previous / Next row)

```
SELECT
  account_id,
  amount,
  LAG(amount, 1) OVER (PARTITION BY account_id ORDER BY timestamp) AS prev_amount
FROM transactions;
```

pandas Equivalent

```
# shift(1) for LAG, shift(-1) for LEAD
df["prev_amount"] = df.sort_values("timestamp").groupby("account_id")["amount"].shift(1)
```

8. `CASE WHEN` (Conditional Column Assignment)

SQL

```
SELECT
  transaction_id,
  amount,
  CASE
    WHEN amount ≥ 50000 THEN 'HIGH'
    WHEN amount ≥ 10000 THEN 'MEDIUM'
    ELSE 'LOW'
  END AS risk_category
FROM transactions;
```

pandas (Vectorized with `np.select`)

```
import numpy as np

conditions = [
    df["amount"] >= 50000,
    df["amount"] >= 10000
]
choices = ["HIGH", "MEDIUM"]

df["risk_category"] = np.select(conditions, choices, default="LOW")
```

4. Quick Summary Checklist for Interviews

- `SELECT col` $\implies$ `df['col']` or `df[['a', 'b']]`
- `WHERE` $\implies$ `df[(df['col'] > x) & (df['col2'] == y)]`
- `GROUP BY` $\implies$ `df.groupby('col').agg(...)`
- `HAVING` $\implies$ filter the aggregated DataFrame output
- `JOIN` $\implies$ `pd.merge(df1, df2, on='key', how='inner|left|outer')`
- `UNION ALL` $\implies$ `pd.concat([df1, df2], axis=0)`
- `SUM() OVER(PARTITION BY ...)` $\implies$ `df.groupby('col')['val'].cumsum()`
- `CASE WHEN` $\implies$ `np.select(conditions, choices, default=...)`

CHAPTER 06 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- ``SELECT col`` $\implies$ ``df['col']`` or ``df[['a', 'b']]``
- ``WHERE`` $\implies$ ``df[(df['col'] > x) & (df['col2'] == y)]``
- ``GROUP BY`` $\implies$ ``df.groupby('col').agg(...)``

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 07

The Complete Python Data Analysis Workflow: NumPy to pandas to Matplotlib

PRIORITY ● High Priority	DIFFICULTY Advanced Core	EST. TIME 35–45 min	FREQUENCY Very High (85%+)
--	--	---	--

🚀 The Complete Python Data Analysis Workflow: NumPy to pandas to Matplotlib

1. The Real-World Engineering Pipeline

In production banking backends and analytics microservices, raw transactional data often flows from database queries, CSV dumps, or message queues into an analytical pipeline:

```
flowchart TD
    Raw["1. Raw Data Source (transactions.csv / SQL Dump)"] --> Load["2. Load into pandas DataFrame (pd.read_csv / pd.read_sql)"]
    Load --> Clean["3. Data Cleaning (Impute NaNs, drop duplicates, parse timestamps)"]
    Clean --> Vectorize["4. NumPy Vectorization (Z-score anomaly detection, thresholding)"]
    Vectorize --> Agg["5. pandas Aggregation (Split-Apply-Combine groupby & pivots)"]
    Agg --> Viz["6. Matplotlib Visualization (Multi-panel dashboard figure)"]
    Viz --> Output["7. Export Clean Data & Executive Dashboard PNG"]
```

2. Step-by-Step Code Walkthrough (Banking Transaction Analytics)

Step 1: Load Data into pandas

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("01-PYTHON/transactions.csv")
print(f"Loaded {len(df)} transactions. Shape: {df.shape}")
```

Step 2: Clean and Impute Missing Values

```
# Check missing counts
print(df.isna().sum())

# Impute numerical missing amounts with the median to avoid skewing:
median_amount = df["amount"].median()
df["amount"] = df["amount"].fillna(median_amount)

# Impute categorical missing values:
df["merchant_category"] = df["merchant_category"].fillna("UNKNOWN")

# Convert timestamp strings to true pandas DateTime:
df["timestamp"] = pd.to_datetime(df["timestamp"])
df["date"] = df["timestamp"].dt.date
```

Step 3: Beginner Analytics (Key KPI Summary)

```
total_volume = df["amount"].sum()
avg_ticket = df["amount"].mean()
max_txn = df["amount"].max()
status_counts = df["status"].value_counts()

print(f"Total Settled: ₹{total_volume:,.2f}")
print(f"Avg Ticket: ₹{avg_ticket:,.2f}")
print(f"Status Distribution:\n{status_counts}")
```

Step 4: Intermediate Analytics (GroupBy Spend by Category)

```
category_summary = df.groupby("merchant_category").agg(
    volume=("amount", "sum"),
    txn_count=("transaction_id", "count"),
    avg_ticket=("amount", "mean")
).sort_values("volume", ascending=False)

print(category_summary)
```

Step 5: Advanced Anomaly Detection Using NumPy Vectorization

```
# Statistical Outlier Detection: Z-Score > 3 (Mean + 3 * StdDev)
mean_val = np.mean(df["amount"])
std_val = np.std(df["amount"])
anomaly_cutoff = mean_val + (3 * std_val)

# Vectorized boolean mask
anomalies = df[df["amount"] > anomaly_cutoff]
print(f"Anomaly Cutoff: > ₹{anomaly_cutoff:,.2f}")
print(f"Flagged Suspicious Transactions: {len(anomalies)}")
```

Step 6: Visual Dashboard Generation with Matplotlib OO API

```
fig, axes = plt.subplots(2, 2, figsize=(14, 9))
fig.suptitle("IDFC FIRST Bank – Transaction Health Dashboard", fontsize=16, fontweight="bold",
color="#9b1c1c")

# 1. Daily Volume Trend (Line Chart)
daily = df.groupby("date")["amount"].sum()
axes[0, 0].plot(daily.index, daily.values, color="#9b1c1c", linewidth=2)
axes[0, 0].set_title("Daily Transaction Volume (₹)")
axes[0, 0].grid(True, linestyle="--", alpha=0.5)

# 2. Category Spend (Bar Chart)
axes[0, 1].bar(category_summary.index, category_summary["volume"] / 1000, color="#2b6cb0")
axes[0, 1].set_title("Volume by Category (₹ '000)")
axes[0, 1].tick_params(axis="x", rotation=30)

# 3. Success vs Failure (Pie Chart)
axes[1, 0].pie(status_counts.values, labels=status_counts.index, autopct="%1.1f%", colors=
["#38a169", "#e53e3e", "#dd6b20"])
axes[1, 0].set_title("Transaction Status Ratio")

# 4. Distribution with Anomaly Cutoff (Histogram)
axes[1, 1].hist(df["amount"], bins=30, color="#4a5568", edgecolor="white")
axes[1, 1].axvline(anomaly_cutoff, color="red", linestyle="--", label=f"Anomaly (>₹
{int(anomaly_cutoff):,})")
axes[1, 1].set_title("Transaction Amount Distribution")
axes[1, 1].legend()

plt.tight_layout()
fig.savefig("01-PYTHON/banking_analytics_dashboard.png", dpi=300)
plt.close(fig)
```

3. How to Run the Live Pipeline

You can run this full pipeline anytime directly in your terminal:

```
python3 01-PYTHON/banking-analytics-project.py
```

It loads [01-PYTHON/transactions.csv](#), computes all analytics, and updates [01-PYTHON/banking_analytics_dashboard.png](#).

 CHAPTER 07 SUMMARY & REVISION CHECKLIST **WHAT TO REMEMBER**

- Core fundamentals and architectural mechanics of The Complete Python Data Analysis Workflow: NumPy to pandas to Matplotlib
- Production banking scenarios and transaction reliability
- High-frequency technical interview questions and follow-ups

 CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 08

Last-Minute Python Data Ecosystem Cheatsheet

PRIORITY

● High Priority

DIFFICULTY

Advanced Core

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

⚡ Last-Minute Python Data Ecosystem Cheatsheet

🐍 Python in 5 Minutes

1. Types & Mutability

- **Immutable (Cannot change in-place):** `int`, `float`, `str`, `tuple`, `frozenset`, `bytes`.
- **Mutable (In-place changes):** `list`, `dict`, `set`, `bytearray`.
- **List vs Tuple:** List mutable hai (`[1, 2]`); Tuple immutable hai (`(1, 2)`) - faster, hashable, can be dict keys).
- **Dict vs Set:** Dict key-value map hai; Set unique keys only hai (both $O(1)$ avg lookups).

2. Copying & References

- `b = a` $\implies$ Same pointer (mutating `b` mutates `a`).
- `shallow = a.copy()` $\implies$ New outer container, shared nested objects.
- `deep = copy.deepcopy(a)` $\implies$ Completely independent recursive clone.

3. Comprehensions & Packing

- `[x*2 for x in arr if x > 0]` (List) | `{k: v for k, v in pairs}` (Dict)
- `def f(*args, **kwargs):` `*args` captures tuple, `**kwargs` captures dictionary.

4. Generators, Decorators & Context Managers

- **Generator** (`yield`): Lazy evaluation, produces 1 item at a time in $O(1)$ memory.
- **Decorator** (`@func`): Wraps another function to add logging/auth without altering code.
- **Context Manager** (`with`): Guarantees `__exit__()` cleanup on DB connections/files.

1 2 3 4 NumPy in 3 Minutes

1. ndarray Anatomy

- **Contiguous memory:** C-style contiguous buffer of homogeneous bytes (unlike scattered Python lists).

- **shape** : Dimension tuple (e.g., `(2, 3)` = 2 rows, 3 cols).
- **axis=0** : Down rows (column-wise). **axis=1** : Across cols (row-wise).
- **dtype** : Data type (`int64`, `float32`).

2. Slicing & Reshape

- **Slices are Views**: Modifying `arr[1:3]` modifies the original array! Use `.copy()` if you need independence.
- **reshape(3, -1)** : `-1` auto-calculates remaining dimension.
- **flatten()** : Always returns a copy. **ravel()** : Returns a view when possible.

3. Broadcasting & Vectorization

- **Broadcasting Rules**: Dimensions match or one of them is 1 (stretches along size 1).
- **Vectorization**: Replaces slow Python loops with hardware SIMD C-level CPU instructions (40x–80x faster).



pandas in 5 Minutes

1. Core Structures

- **Series**: 1D labeled array. **DataFrame**: 2D labeled table of Series.

2. Selection: `loc` vs `iloc`

- **df.loc['row', 'col']** : **Label-based** (inclusive of end boundary).
- **df.iloc[0:5, 0:2]** : **Integer-position based** (0-indexed, exclusive of end boundary).

3. Filtering & Cleaning

- Multiple filters: `df[(df['amt'] > 1000) & (df['status'] == 'SUCCESS')]` (use `&` and `|` with parentheses).
- Missing Data: `df.isna().sum()` | `df.dropna()` | `df['amt'].fillna(df['amt'].median())`.
- Duplicates: `df.drop_duplicates(subset=['txn_id'])`.

4. GroupBy & Merging

- **Split-Apply-Combine**: `python df.groupby('category')['amt'].agg(['sum', 'mean', 'count']).reset_index()`
- **SQL Merges**: `python pd.merge(df1, df2, on='cust_id', how='left')` # SQL: LEFT JOIN
`pd.concat([df1, df2], axis=0)` # SQL: UNION ALL
- **Avoid `iterrows()`** : Vectorize with `np.select` or list comprehensions instead of loops.



Matplotlib in 2 Minutes

1. Anatomy: Figure vs Axes

- **Figure:** The overall canvas/window.
- **Axes:** The actual individual plot/graph (x-axis, y-axis, data points, title).

2. Object–Oriented Interface (Recommended)

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(x, y, color="#9b1c1c", label="Trend")
ax.set_title("IDFC Transaction Volume")
ax.set_xlabel("Date")
ax.set_ylabel("Volume (₹)")
ax.grid(True, linestyle="--")
ax.legend()
plt.tight_layout()
fig.savefig("chart.png", dpi=300)
plt.close(fig)
```

3. Chart Types

- **Line** (`ax.plot`): Time-series trends.
- **Bar** (`ax.bar`): Categorical comparisons.
- **Histogram** (`ax.hist`): Numerical distribution.
- **Scatter** (`ax.scatter`): Relationship / correlations between two variables.



CHAPTER 08 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Immutable (Cannot change in-place): ``int`, `float`, `str`, `tuple`, `frozenset`, `bytes``.
- Mutable (In-place changes): ``list`, `dict`, `set`, `bytearray``.
- List vs Tuple: List mutable hai (``[1, 2]``); Tuple immutable hai (``(1, 2)`` - faster, hashable, can be dict keys).



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases